

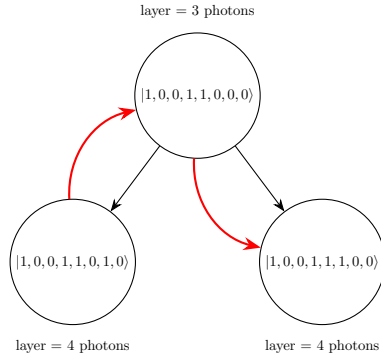
Compute Slos incrementally from previous amplitudes

Grégoire Leboucher

May 2025

1 Introduction

SLOS, which stands for Strong Linear Optical Simulation, computes the complete quantum state obtained at the output of a photonic circuit. As SLOS can only compute Fock states, if one needs to simulate the output of a superposition of Fock states, one needs to compute through SLOS all of the corresponding Fock states. For a circuit with n photons and m modes, there are potentially $\binom{m+n-1}{n}$ different Fock states to compute. The current version of SLOS recomputes the entire tree for every Fock state and does not take advantage of the similarities between the different states. Let us, for instance, consider a circuit with 8 modes and 4 photons and the following input state: $\frac{1}{\sqrt{2}} |1, 0, 0, 1, 0, 0, 1, 1, 0, 0\rangle + \frac{1}{\sqrt{2}} |1, 0, 0, 1, 0, 0, 1, 0, 1, 0\rangle$. A new feature in SLOS could compute the first Fock state, record the amplitudes, and notice that the second input state can be obtained by switching the photon on the seventh mode of the first Fock state to the sixth mode. The algorithm would then find the amplitudes if we remove the seventh mode photon and then compute the layer to add a photon in the sixth mode.



2 SLOS current function

As mentioned earlier in this document, SLOS can be seen as a tree, each layer of the tree representing a number of photons. Thus, a layer will always transform the amplitudes of a state with k photons into a state with $k + 1$ photons. For each layer, the following three variables are left unchanged:

- Sources: Matches the different input Fock states with the number of operations. If the no bunching mode is activated, only the Fock states leading to a no bunching output state are considered. For example, the state $|1110000000\rangle$ will be repeated only seven times as the output photon can only be in positions 4 to 10.
- Modes: Defines which operations must be computed in which state. This plus the index of the new input photon will allow you to select the right elements of the unitary to be multiplied by the input amplitudes transformed.
- Finally, the destinations vector scatter adds the amplitudes that correspond to the same output Fock state.

All of this can be summarized in the following equation:

$$amplitudes = D \times U \times S \times previous_amplitudes$$

with :

- S : Source tensor (size: (number of operations, input size))
- U: Diagonal tensor composed of elements from the unitary matrix (size: (number of operations, number of operations))
- D : Destination tensor (equivalent to the scatter add operation, size: (output size, number of operations))

The resulting tensor is thus $C = D \times U \times S$ of size: (output size, input size)

3 The backward layer

As we manipulate circuits that have largely more modes than photons, the output size is always bigger than the input size. This means that, when we have some information about the amplitudes and the tensor C , it is possible to obtain the previous amplitudes by calculating the pseudo-inverse of C , $C^\dagger$,

$$previous_amplitudes = C^\dagger \times amplitudes$$

However, computing the pseudo-inverse is a very costly operation (for a matrix of size $N \times M$, the cost is $O(N \times M^2)$, and as the output size is $\binom{m+n-1}{n}$, ($\binom{m}{n}$ with the no bunching mode) the computing time rockets with the number

of modes m and the number of photons n . Hence, computing one backward layer is often longer than calculating the final amplitudes with the current SLOS.

As mentioned earlier, the three different variables: sources, modes and destinations only depend on the number of photons and whether the no bunching mode is activated or not. This means that the tensors D and S are unchanged if we modify the input state or the unitary. This means that, as long as the unitary do not change, we can store all of those pseudo-inverse tensors, the calculus of the amplitudes then turns out to be only the result of two tensor products.

This idea was used in the Boson Sampler, where the input data is encoded on the amplitudes of the input state given to the quantum circuit. If we don't train the Boson Sampler, the parameters and thus the unitary tensor do not change. We compute those pseudo-inverse tensors only once at the beginning which allow us to decrease on the overall the inference time.